

# Module 04: Feature Scaling & Data Transformation

**Lecturer:** Abdan Hafidz

**Topic:** The Geometry of Data Preprocessing

---

## 1 Introduction: Comparing Apples to Elephants

Imagine you are training a model to predict a person's health. You have two input features:

- **Age:** Range 0 to 100 years.
- **Income:** Range \$0 to \$1,000,000,000.

To a human, we know these are different units. But to a Machine Learning algorithm (like KNN or Gradient Descent), these are just numbers. The algorithm calculates distance using Euclidean geometry:

$$\text{Distance}^2 = (\text{Age}_1 - \text{Age}_2)^2 + (\text{Income}_1 - \text{Income}_2)^2$$

Because Income is in the *billions* and Age is only in the *tens*, the Income squared term will mathematically drown out the Age term. The model will essentially ignore Age completely. This is why we must perform **Feature Scaling**: to bring all distinct features onto a level playing field.

Lecturer's Insight: Scaling vs. Transformation

**Scaling** changes the **Range** (e.g., 0 to 1). It is like zooming in or out on a photo.

**Transformation** changes the **Shape**. It is like using a fisheye lens or warping the photo to correct a distortion (e.g., turning a skewed distribution into a bell curve).

## 2 Min-Max Scaling (Normalization)

This is the most intuitive approach. We treat the data as a bounding box and shrink it to fit within a specific range, typically  $[0, 1]$ .

### 2.1 The Formula

$$X_{\text{scaled}} = \frac{X - X_{\min}}{X_{\max} - X_{\min}}$$

**Pros:** Preserves the exact shape of the original distribution. Excellent for Neural Networks.

**Cons (The Outlier Trap):** If you have a single outlier (e.g., a billionaire), the denominator becomes massive, crushing all "normal" data into a tiny cluster near 0.

### 3 Standardization (Z-Score Scaling)

Standardization transforms the data so that it has a Mean ( $\mu$ ) of 0 and a Standard Deviation ( $\sigma$ ) of 1.

#### 3.1 The Formula

$$Z = \frac{X - \mu}{\sigma}$$

**When to use?** When your algorithm assumes data is Gaussian (e.g., Logistic Regression). It handles outliers slightly better than Min-Max but is still sensitive to them because the Mean itself is sensitive.

### 4 The Robust Scaler

What if your data is full of outliers? We need metrics that ignore them: the **Median** and the **IQR**.

#### 4.1 The Formula

$$X_{robust} = \frac{X - \text{Median}}{IQR}$$

By scaling using the Interquartile Range (the middle 50% of data), outliers are left far out in the distance, but they do not distort the scale of the "normal" data.

## 5 Visual Comparison of Techniques

Let's visualize how these three techniques handle a dataset with one extreme outlier.

**Scenario:** Most people have wealth between \$10k–\$100k. One person has \$10,000k (Outlier).

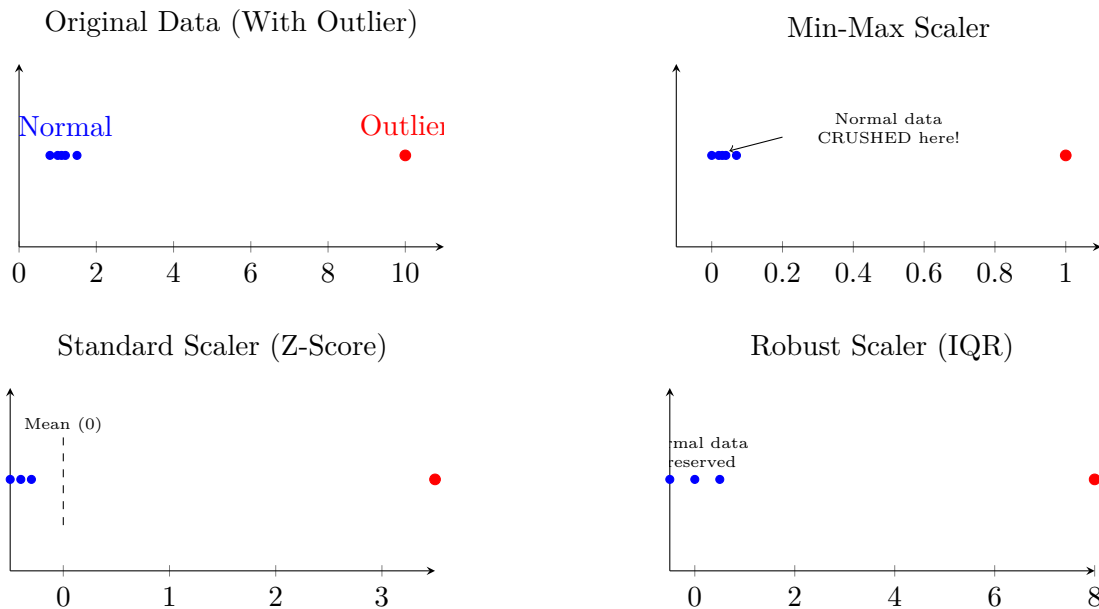


Figure 1: **Effect of Outliers on Scaling.** Notice how in Min-Max Scaling (Top Right), the outlier at 1.0 forces all normal data to squash into a tiny cluster near 0.0. The Robust Scaler (Bottom Right) keeps the normal data spread out, allowing the model to see differences between them.

## 6 Data Transformation: Reshaping the Clay

Sometimes, scaling isn't enough. Many statistical models (Linear Regression, Naive Bayes) fundamentally assume that the errors are **Normally Distributed** (Gaussian). If your data is heavily **Skewed** (e.g., Income data), the model will struggle.

### 6.1 Log Transformation (The Right-Skew Killer)

Right-skewed data has a "long tail" to the right. Logarithm compresses large values.

$$X_{new} = \log(X + 1)$$

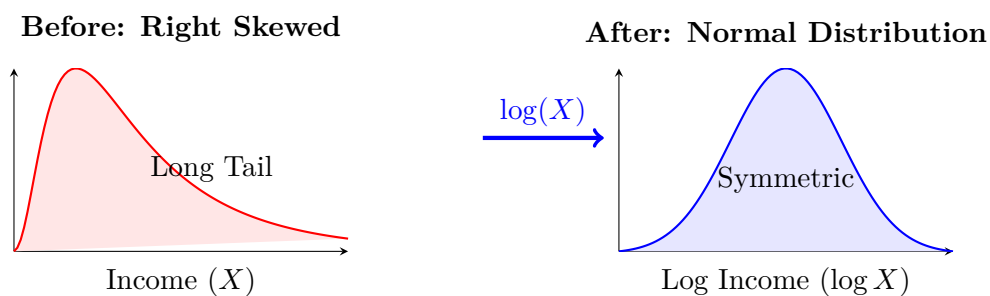


Figure 2: The Magic of Log Transformation. It takes a stretched, skewed distribution (Left) and compresses the tail to form a clean, symmetric Bell Curve (Right).

## 7 Summary Table & Implementation

| Technique       | Best For...                | Handling Outliers          |
|-----------------|----------------------------|----------------------------|
| Min-Max Scaler  | Neural Networks, Images    | <b>Poor</b> (Crushed)      |
| Standard Scaler | Linear/Logistic Regression | Moderate (Skewed Mean)     |
| Robust Scaler   | Data with Extreme Outliers | <b>Excellent</b> (Ignored) |
| Log Transform   | Right-Skewed Data          | N/A (Changes Shape)        |
| Power Transform | Left-Skewed Data           | N/A (Changes Shape)        |

### Python Implementation (Complete):

```
1 import numpy as np
2 import pandas as pd
3 from sklearn.preprocessing import MinMaxScaler, StandardScaler, RobustScaler
4 from sklearn.preprocessing import PowerTransformer
5
6 # 1. SCALING (Changing Range)
7 # -----
8 # Min-Max: Use for Deep Learning / Image Data
9 scaler_mm = MinMaxScaler()
10 X_mm = scaler_mm.fit_transform(df[['income']])
11
12 # Standard: Use for Linear/Logistic Regression
13 scaler_std = StandardScaler()
14 X_std = scaler_std.fit_transform(df[['income']])
15
16 # Robust: Use if you have crazy outliers (e.g., Billionaires)
17 scaler_rob = RobustScaler()
18 X_rob = scaler_rob.fit_transform(df[['income']])
19
20 # 2. TRANSFORMATION (Changing Shape)
21 # -----
22 # Manual Log Transform (Good for Right Skew / High Income)
23 df['log_income'] = np.log1p(df['income'])
24
25 # Automatic Power Transform (Yeo-Johnson)
26 # Finds the best shape mathematically to make data Normal
27 pt = PowerTransformer(method='yeo-johnson')
28 df['income_normal'] = pt.fit_transform(df[['income']])
```